

Bài tập về nhà — Bài TH 01 (Vyper)

Nội dung theo mục **4. Bài tập về nhà** trong `Bai01.pdf`.

Bài	File	Nội dung
1	<code>Bai1_StorageInt.vy</code>	int128 store / retrieve
2	<code>Bai2_DepositWithdraw.vy</code>	deposit / balances / withdraw
3	<code>Bai3_Voting.vy</code>	bỏ phiếu Tỳ-Tèo-Mai
4	<code>Bai4_Contacts.vy</code>	struct danh bạ
5	<code>Bai5_Crowdfunding.vy</code>	gây quỹ 5 ETH / 10 phút
6	<code>Bai6_DocumentNotary.vy</code>	công chứng hash + chữ ký

Cách làm: chạy từng cell ghi file → cell biên dịch bên dưới. Có thể upload `.vy` lên Remix (plugin Vyper) để Deploy.

Setup nhanh (Windows)

Chọn kernel `.venv` của project. Nếu chưa có `vyper`:

```
In [1]: %pip install -q vyper
```

```
[skip pip] vyper ready
```

```
In [2]: from pathlib import Path
from vyper.compiler import compile_code

def compile_vy(filename: str):
    src = Path(filename).read_text(encoding="utf-8")
    print("File:", Path(filename).resolve())
    print("-" * 50)
    try:
        result = compile_code(src, output_formats=["abi", "bytecode"])
        abi = result.get("abi") or []
        fns = [x.get("name") for x in abi if isinstance(x, dict) and x.get("type")]
        bytecode = str(result.get("bytecode") or "")
        print("[OK] Biên dịch thành công")
        print("ABI functions:", ", ".join(fns) if fns else "(không có)")
        print("Bytecode (64 ký tu đầu):", bytecode[:64], ("..." if len(bytecode) >
    except Exception as e:
        print("[FAIL]", type(e).__name__)
        print(e)
```

Bài 1: Kiểu dữ liệu & hàm cơ bản

Yêu cầu:

- Lưu một số nguyên (`int128`) vào biến trạng thái.
- Lấy lại giá trị đã lưu.

Hướng dẫn: Viết 2 hàm: `store` để lưu, `retrieve` để trả về giá trị.

File: `Bai1_StorageInt.vy`

```
In [3]: from pathlib import Path

source = r'''# @version ^0.4.0

"""Bài 1: Lưu trữ số nguyên (int128)."""

stored_value: int128

@external
def store(num: int128):
    """Lưu một số nguyên vào biến trạng thái."""
    self.stored_value = num

@external
@view
def retrieve() -> int128:
    """Trả về giá trị đã lưu."""
    return self.stored_value
...

Path("Bai1_StorageInt.vy").write_text(source, encoding="utf-8")
print("Da ghi Bai1_StorageInt.vy")
print(source)
```

Da ghi Bai1_StorageInt.vy

```
# @version ^0.4.0
```

```
"""Bài 1: Lưu trữ số nguyên (int128)."""
```

```
stored_value: int128
```

```
@external
```

```
def store(num: int128):
```

```
    """Lưu một số nguyên vào biến trạng thái."""
```

```
    self.stored_value = num
```

```
@external
```

```
@view
```

```
def retrieve() -> int128:
```

```
    """Trả về giá trị đã lưu."""
```

```
    return self.stored_value
```

```
In [4]: compile_vy("Bai1_StorageInt.vy")
```

File: D:\Study\IS355\TH\w1\BTTH_VeNha\Bai1_StorageInt.vy

[OK] Bien dịch thành công

ABI functions: store, retrieve

Bytecode (64 ký tự đầu): 0x61006b61000f60003961006b6000f35f3560e01c60026001821660011b6100 ...

Bài 2: Thanh toán ETH & quản lý số dư

Yêu cầu:

- Người dùng gửi ETH vào hợp đồng (`deposit`).
- Lưu trữ số dư của từng địa chỉ.
- Cho phép người dùng rút tiền của mình (`withdraw`).

Hướng dẫn: 3 hàm: `deposit()`, `get_balance(addr)` (balances), `withdraw()`.

File: Bai2_DepositWithdraw.vy

```
In [5]: from pathlib import Path

source = r'''# @version ^0.4.0

"""Bài 2: Gửi/rút ETH, quản lý số dư theo địa chỉ."""

balances: public(HashMap[address, uint256])

@external
@payable
def deposit():
    """Người dùng gửi ETH vào hợp đồng."""
    assert msg.value > 0, "must send ETH"
    self.balances[msg.sender] += msg.value

@external
@view
def get_balance(addr: address) -> uint256:
    """Kiểm tra số dư của một địa chỉ (balances)."""
    return self.balances[addr]

@external
def withdraw():
    """Rút toàn bộ số dư của người gọi."""
    amount: uint256 = self.balances[msg.sender]
    assert amount > 0, "no balance"
    self.balances[msg.sender] = 0
    raw_call(msg.sender, b"", value=amount)
...'''
```

```
Path("Bai2_DepositWithdraw.vy").write_text(source, encoding="utf-8")
print("Da ghi Bai2_DepositWithdraw.vy")
print(source)
```

```
Da ghi Bai2_DepositWithdraw.vy
# @version ^0.4.0
```

```
"""Bài 2: Gửi/rút ETH, quản lý số dư theo địa chỉ."""
```

```
balances: public(HashMap[address, uint256])
```

```
@external
@payable
def deposit():
    """Người dùng gửi ETH vào hợp đồng."""
    assert msg.value > 0, "must send ETH"
    self.balances[msg.sender] += msg.value
```

```
@external
@view
def get_balance(addr: address) -> uint256:
    """Kiểm tra số dư của một địa chỉ (balances)."""
    return self.balances[addr]
```

```
@external
def withdraw():
    """Rút toàn bộ số dư của người gọi."""
    amount: uint256 = self.balances[msg.sender]
    assert amount > 0, "no balance"
    self.balances[msg.sender] = 0
    raw_call(msg.sender, b"", value=amount)
```

```
In [6]: compile_vy("Bai2_DepositWithdraw.vy")
```

```
File: D:\Study\IS355\TH\w1\BTTH_VeNha\Bai2_DepositWithdraw.vy
```

```
-----
[OK] Bien dịch thành công
```

```
ABI functions: deposit, get_balance, withdraw, balances
```

```
Bytecode (64 ký tự dấu): 0x61020c6100116100003961020c610000f35f3560e01c60026005820660011b ...
```

Bài 3: Cấu trúc điều khiển & HashMap

Yêu cầu:

- Có danh sách ứng cử viên (mặc định: Tý, Tèo, Mai).
- Mỗi địa chỉ chỉ được bỏ phiếu một lần.
- Xác định người thắng cuộc.

Hướng dẫn: Hàm `vote(name_candidate)` và `getWinner()`.

File: Bai3_Voting.vy

In [7]: `from pathlib import Path`

```
source = r'''# @version ^0.4.0

"""Bài 3: Bỏ phiếu đơn giản - ứng viên Tý, Tèo, Mai."""

candidates: DynArray[String[32], 3]
votes: HashMap[String[32], uint256]
has_voted: HashMap[address, bool]

@deploy
def __init__():
    self.candidates.append("Ty")
    self.candidates.append("Teo")
    self.candidates.append("Mai")

@internal
@view
def _is_candidate(name: String[32]) -> bool:
    for c: String[32] in self.candidates:
        if c == name:
            return True
    return False

@external
def vote(name_candidate: String[32]):
    """Mỗi địa chỉ chỉ được bỏ phiếu một lần."""
    assert not self.has_voted[msg.sender], "already voted"
    assert self._is_candidate(name_candidate), "invalid candidate"
    self.has_voted[msg.sender] = True
    self.votes[name_candidate] += 1

@external
@view
def getWinner() -> String[32]:
    """Trả về ứng viên có số phiếu cao nhất."""
    best_name: String[32] = self.candidates[0]
    best_votes: uint256 = self.votes[best_name]
    for c: String[32] in self.candidates:
        if self.votes[c] > best_votes:
            best_votes = self.votes[c]
            best_name = c
    return best_name

@external
@view
def getVotes(name_candidate: String[32]) -> uint256:
    return self.votes[name_candidate]
...

Path("Bai3_Voting.vy").write_text(source, encoding="utf-8")
print("Da ghi Bai3_Voting.vy")
print(source)
```

```

Da ghi Bai3_Voting.vy
# @version ^0.4.0

""""Bài 3: Bỏ phiếu đơn giản - ứng viên Tý, Tèo, Mai."""

candidates: DynArray[String[32], 3]
votes: HashMap[String[32], uint256]
has_voted: HashMap[address, bool]

@deploy
def __init__():
    self.candidates.append("Ty")
    self.candidates.append("Teo")
    self.candidates.append("Mai")

@internal
@view
def _is_candidate(name: String[32]) -> bool:
    for c: String[32] in self.candidates:
        if c == name:
            return True
    return False

@external
def vote(name_candidate: String[32]):
    """"Mỗi địa chỉ chỉ được bỏ phiếu một lần."""
    assert not self.has_voted[msg.sender], "already voted"
    assert self._is_candidate(name_candidate), "invalid candidate"
    self.has_voted[msg.sender] = True
    self.votes[name_candidate] += 1

@external
@view
def getWinner() -> String[32]:
    """"Trả về ứng viên có số phiếu cao nhất."""
    best_name: String[32] = self.candidates[0]
    best_votes: uint256 = self.votes[best_name]
    for c: String[32] in self.candidates:
        if self.votes[c] > best_votes:
            best_votes = self.votes[c]
            best_name = c
    return best_name

@external
@view
def getVotes(name_candidate: String[32]) -> uint256:
    return self.votes[name_candidate]

```

```
In [8]: compile_vy("Bai3_Voting.vy")
```

File: D:\Study\IS355\TH\w1\BTTH_VeNha\Bai3_Voting.vy

[OK] Bien dịch thành công

ABI functions: vote, getWinner, getVotes

Bytecode (64 ký tự đầu): 0x3461011b575f546002811161011b5760026040527f5479000000000000
00000 ...

Bài 4: Struct & tra cứu dữ liệu

Yêu cầu:

- Lưu danh sách liên hệ: tên, số điện thoại (struct).
- Thêm liên hệ mới.
- Tra cứu số điện thoại theo tên.

Hướng dẫn: Hàm `addContract(name, phone)` và `getPhone(name)` (theo đề).

File: Bai4_Contacts.vy

```
In [9]: from pathlib import Path

source = r'''# @version ^0.4.0

"""Bài 4: Quản lý danh bạ (struct + tra cứu theo tên)."""

struct Contact:
    name: String[64]
    phone: String[32]

contacts: HashMap[String[64], Contact]

@external
def addContract(name: String[64], phone: String[32]):
    """Thêm liên hệ mới (theo đề: addContract)."""
    assert len(name) > 0, "empty name"
    self.contacts[name] = Contact(name=name, phone=phone)

@external
@view
def getPhone(name: String[64]) -> String[32]:
    """Tra cứu số điện thoại theo tên."""
    assert len(self.contacts[name].name) > 0, "not found"
    return self.contacts[name].phone
...

Path("Bai4_Contacts.vy").write_text(source, encoding="utf-8")
print("Đã ghi Bai4_Contacts.vy")
print(source)
```

```

Da ghi Bai4_Contacts.vy
# @version ^0.4.0

"""Bài 4: Quản lý danh bạ (struct + tra cứu theo tên)."""

struct Contact:
    name: String[64]
    phone: String[32]

contacts: HashMap[String[64], Contact]

@external
def addContract(name: String[64], phone: String[32]):
    """Thêm liên hệ mới (theo đề: addContract)."""
    assert len(name) > 0, "empty name"
    self.contacts[name] = Contact(name=name, phone=phone)

@external
@view
def getPhone(name: String[64]) -> String[32]:
    """Tra cứu số điện thoại theo tên."""
    assert len(self.contacts[name].name) > 0, "not found"
    return self.contacts[name].phone

```

In [10]: `compile_vy("Bai4_Contacts.vy")`

File: D:\Study\IS355\TH\w1\BTTH_VeNha\Bai4_Contacts.vy

[OK] Biên dịch thành công

ABI functions: addContract, getPhone

Bytecode (64 ký tự dấu): 0x61026561001161000039610265610000f35f3560e01c60026003820660011b ...

Bài 5: Crowdfunding (logic + biến môi trường)

Yêu cầu:

- Mục tiêu 5 ETH, deadline 10 phút.
- Người dùng quyên góp bằng ETH (`contribute`).
- Đạt mục tiêu → chủ dự án rút tiền; không đạt sau deadline → người đóng góp rút lại.

Hướng dẫn: Dùng `msg.value`, `block.timestamp`, `raw_call` để chuyển ETH.

File: `Bai5_Crowdfunding.vy`

In [11]: `from pathlib import Path`

```
source = r'''# @version ^0.4.0
```

```
"""Bài 5: Crowdfunding - mục tiêu 5 ETH, deadline 10 phút."""
```

```

owner: public(address)
goal: public(uint256)
deadline: public(uint256)
totalRaised: public(uint256)
contributions: public(HashMap[address, uint256])
ownerWithdrawn: public(bool)

@deploy
def __init__():
    self.owner = msg.sender
    self.goal = 5 * 10**18 # 5 ETH
    self.deadline = block.timestamp + 10 * 60 # 10 phút

@external
@payable
def contribute():
    """Người dùng quyền góp ETH."""
    assert block.timestamp < self.deadline, "campaign ended"
    assert msg.value > 0, "must send ETH"
    self.contributions[msg.sender] += msg.value
    self.totalRaised += msg.value

@external
def withdraw():
    """
    - Đạt mục tiêu: chủ dự án rút toàn bộ quỹ.
    - Không đạt và đã hết hạn: người đóng góp rút lại phần của mình.
    """
    if self.totalRaised >= self.goal:
        assert msg.sender == self.owner, "only owner"
        assert not self.ownerWithdrawn, "already withdrawn"
        self.ownerWithdrawn = True
        amount: uint256 = self.balance
        raw_call(self.owner, b"", value=amount)
    else:
        assert block.timestamp >= self.deadline, "still ongoing"
        amount: uint256 = self.contributions[msg.sender]
        assert amount > 0, "nothing to refund"
        self.contributions[msg.sender] = 0
        raw_call(msg.sender, b"", value=amount)
    ...

Path("Bai5_Crowdfunding.vy").write_text(source, encoding="utf-8")
print("Đã ghi Bai5_Crowdfunding.vy")
print(source)

```

```

Da ghi Bai5_Crowdfunding.vy
# @version ^0.4.0

""""Bài 5: Crowdfunding - mục tiêu 5 ETH, deadline 10 phút."""

owner: public(address)
goal: public(uint256)
deadline: public(uint256)
totalRaised: public(uint256)
contributions: public(HashMap[address, uint256])
ownerWithdrawn: public(bool)

@deploy
def __init__():
    self.owner = msg.sender
    self.goal = 5 * 10**18 # 5 ETH
    self.deadline = block.timestamp + 10 * 60 # 10 phút

@external
@payable
def contribute():
    """"Người dùng quyền góp ETH."""
    assert block.timestamp < self.deadline, "campaign ended"
    assert msg.value > 0, "must send ETH"
    self.contributions[msg.sender] += msg.value
    self.totalRaised += msg.value

@external
def withdraw():
    """"
    - Đạt mục tiêu: chủ dự án rút toàn bộ quỹ.
    - Không đạt và đã hết hạn: người đóng góp rút lại phần của mình.
    """
    if self.totalRaised >= self.goal:
        assert msg.sender == self.owner, "only owner"
        assert not self.ownerWithdrawn, "already withdrawn"
        self.ownerWithdrawn = True
        amount: uint256 = self.balance
        raw_call(self.owner, b"", value=amount)
    else:
        assert block.timestamp >= self.deadline, "still ongoing"
        amount: uint256 = self.contributions[msg.sender]
        assert amount > 0, "nothing to refund"
        self.contributions[msg.sender] = 0
        raw_call(msg.sender, b"", value=amount)

```

In [12]: `compile_vy("Bai5_Crowdfunding.vy")`

File: D:\Study\IS355\TH\w1\BTTH_VeNha\Bai5_Crowdfunding.vy

[OK] Bien dịch thành công

ABI functions: contribute, withdraw, owner, goal, deadline, totalRaised, contributions, ownerWithdrawn

Bytecode (64 ký tự đầu): 0x3461003757335f55674563918244f400006001554261025881018181106100 ...

Bài 6 (nâng cao): Document Notary

Yêu cầu:

- Đăng ký tài liệu bằng hash (`bytes32`), lưu owner + timestamp, event `DocumentRegistered` .
- Xác thực tài liệu có tồn tại không (`verifyDocument`).
- Chủ sở hữu đính kèm chữ ký (`String[256]`), event `SignatureAdded` .
- Chỉ người đăng ký mới được thêm chữ ký.

Hướng dẫn: Dùng `block.timestamp` và event logging.

File: `Bai6_DocumentNotary.vy`

```
In [13]: from pathlib import Path

source = r'''# @version ^0.4.0

"""Bài 6 (nâng cao): Document Notary - công chứng hash + chữ ký số."""

struct Document:
    doc_hash: bytes32
    owner: address
    timestamp: uint256
    signature: String[256]
    exists: bool

documents: HashMap[bytes32, Document]

event DocumentRegistered:
    doc_hash: indexed(bytes32)
    owner: indexed(address)
    timestamp: uint256

event SignatureAdded:
    doc_hash: indexed(bytes32)
    owner: indexed(address)
    signature: String[256]

@external
def registerDocument(doc_hash: bytes32):
    """Đăng ký tài liệu bằng hash (bytes32)."""
    assert doc_hash != empty(bytes32), "empty hash"
    assert not self.documents[doc_hash].exists, "already registered"
    self.documents[doc_hash] = Document(
        doc_hash=doc_hash,
        owner=msg.sender,
        timestamp=block.timestamp,
        signature="",
        exists=True,
```

```

    )
    log DocumentRegistered(doc_hash=doc_hash, owner=msg.sender, timestamp=block.timestamp)

@external
@view
def verifyDocument(doc_hash: bytes32) -> bool:
    """Kiểm tra tài liệu với hash đã tồn tại chưa."""
    return self.documents[doc_hash].exists

@external
@view
def getDocument(doc_hash: bytes32) -> Document:
    """Lấy thông tin tài liệu đã đăng ký."""
    assert self.documents[doc_hash].exists, "not found"
    return self.documents[doc_hash]

@external
def addSignature(doc_hash: bytes32, signature: String[256]):
    """Chỉ chủ đăng ký mới được gắn chữ ký số."""
    assert self.documents[doc_hash].exists, "not found"
    assert msg.sender == self.documents[doc_hash].owner, "not owner"
    assert len(signature) > 0, "empty signature"
    self.documents[doc_hash].signature = signature
    log SignatureAdded(doc_hash=doc_hash, owner=msg.sender, signature=signature)
    ...

Path("Bai6_DocumentNotary.vy").write_text(source, encoding="utf-8")
print("Da ghi Bai6_DocumentNotary.vy")
print(source)

```

Da ghi Bai6_DocumentNotary.vy

@version ^0.4.0

""""Bài 6 (nâng cao): Document Notary - công chứng hash + chữ ký số.""""

struct Document:

doc_hash: bytes32

owner: address

timestamp: uint256

signature: String[256]

exists: bool

documents: HashMap[bytes32, Document]

event DocumentRegistered:

doc_hash: indexed(bytes32)

owner: indexed(address)

timestamp: uint256

event SignatureAdded:

doc_hash: indexed(bytes32)

owner: indexed(address)

signature: String[256]

@external

def registerDocument(doc_hash: bytes32):

""""Đăng ký tài liệu bằng hash (bytes32).""""

assert doc_hash != empty(bytes32), "empty hash"

assert not self.documents[doc_hash].exists, "already registered"

self.documents[doc_hash] = Document(

doc_hash=doc_hash,

owner=msg.sender,

timestamp=block.timestamp,

signature="",

exists=True,

)

log DocumentRegistered(doc_hash=doc_hash, owner=msg.sender, timestamp=block.timestamp)

@external

@view

def verifyDocument(doc_hash: bytes32) -> bool:

""""Kiểm tra tài liệu với hash đã tồn tại chưa.""""

return self.documents[doc_hash].exists

@external

@view

def getDocument(doc_hash: bytes32) -> Document:

""""Lấy thông tin tài liệu đã đăng ký.""""

assert self.documents[doc_hash].exists, "not found"

return self.documents[doc_hash]

@external

def addSignature(doc_hash: bytes32, signature: String[256]):

""""Chỉ chủ đăng ký mới được gắn chữ ký số.""""

assert self.documents[doc_hash].exists, "not found"

```
assert msg.sender == self.documents[doc_hash].owner, "not owner"
assert len(signature) > 0, "empty signature"
self.documents[doc_hash].signature = signature
log SignatureAdded(doc_hash=doc_hash, owner=msg.sender, signature=signature)
```

In [14]: `compile_vy("Bai6_DocumentNotary.vy")`

File: D:\Study\IS355\TH\w1\BTTH_VeNha\Bai6_DocumentNotary.vy

[OK] Biên dịch thành công

ABI functions: registerDocument, verifyDocument, getDocument, addSignature

Bytecode (64 ký tự đầu): 0x6105fc610011610000396105fc610000f35f3560e01c60026005820660011b ...

Nội dung bài

Nội dung thư mục `BTTH_VeNha` gồm:

- Notebook `Bai01_BTVN.ipynb` (có output biên dịch)
- Các file `.vy` (Bài 1–6)
- (Tuỳ chọn) ảnh Deploy trên Remix

Kết quả: thư mục `BTTH_VeNha` (notebook + các file `.vy` Bài 1–6).

